

NOW

TODAY

NumPy

NOW
TODAY

NOW
NUMPY

NUMPY
PACKAGE



1. Array Creation

- `np.array()`: Create a NumPy array from a list or other iterable.
Example: `arr = np.array([1, 2, 3, 4])`
- `np.zeros()`: Create an array filled with zeros.
Example: `arr = np.zeros((3, 3))`
- `np.ones()`: Create an array filled with ones.
Example: `arr = np.ones((2, 3))`
- `np.arange()`: Create an array with a range of values.
Example: `arr = np.arange(0, 10, 2)`
- `np.linspace()`: Create an array of evenly spaced values over a specified range.
Example: `arr = np.linspace(0, 1, 5)`
- `np.eye()`: Create an identity matrix.
Example: `arr = np.eye(3)`
- `np.random.rand()`: Create an array of random values from a uniform distribution.
Example: `arr = np.random.rand(3, 3)`
- `np.random.randn()`: Create an array of random values from a standard normal distribution.
Example: `arr = np.random.randn(2, 3)`
- `np.random.randint()`: Generate random integers.
Example: `arr = np.random.randint(1, 10, 5)`

- `np.random.choice()`: Generate random samples from a given array.

Example: `arr = np.random.choice([1, 2, 3], size=5)`

2. Array Properties

- `arr.shape`: Get the shape of an array (rows, columns).
Example: `arr.shape`
 - `arr.size`: Get the total number of elements in an array.
Example: `arr.size`
 - `arr.ndim`: Get the number of dimensions (rank) of an array.
Example: `arr.ndim`
 - `arr.dtype`: Get the data type of the elements in an array.
Example: `arr.dtype`
 - `arr.itemsize`: Get the size in bytes of each element of the array.
Example: `arr.itemsize`
 - `arr.T`: Transpose the array (swap rows and columns).
Example: `arr.T`
 - `arr.flatten()`: Flatten a multidimensional array into a 1D array.
Example: `arr.flatten()`
 - `arr.reshape()`: Reshape an array to a different shape.
Example: `arr.reshape(2, 3)`
-

3. Array Manipulation

- `np.concatenate()`: Concatenate arrays along an axis.
Example: `arr_combined = np.concatenate([arr1, arr2], axis=0)`
 - `np.stack()`: Stack arrays along a new axis.
Example: `arr_stacked = np.stack([arr1, arr2], axis=0)`
 - `np.hstack()`: Stack arrays horizontally (column-wise).
Example: `arr_hstack = np.hstack([arr1, arr2])`
 - `np.vstack()`: Stack arrays vertically (row-wise).
Example: `arr_vstack = np.vstack([arr1, arr2])`
 - `np.split()`: Split an array into multiple sub-arrays.
Example: `arr_split = np.split(arr, 3)`
 - `np.append()`: Append values to an array.
Example: `arr_appended = np.append(arr, [5, 6])`
 - `np.insert()`: Insert values into an array at specified indices.
Example: `arr_inserted = np.insert(arr, 2, 10)`
 - `np.delete()`: Delete values from an array by index.
Example: `arr_deleted = np.delete(arr, 1)`
 - `np.resize()`: Resize an array to a new shape.
Example: `arr_resized = np.resize(arr, (2, 6))`
 - `np.repeat()`: Repeat elements of an array.
Example: `arr_repeated = np.repeat(arr, 2)`
 - `np.tile()`: Repeat an array to create a larger array.
Example: `arr_tiled = np.tile(arr, 2)`
-

4. Mathematical Operations

- `np.add()`: Element-wise addition of arrays.
Example: `arr_sum = np.add(arr1, arr2)`
- `np.subtract()`: Element-wise subtraction of arrays.
Example: `arr_diff = np.subtract(arr1, arr2)`
- `np.multiply()`: Element-wise multiplication of arrays.
Example: `arr_prod = np.multiply(arr1, arr2)`
- `np.divide()`: Element-wise division of arrays.
Example: `arr_div = np.divide(arr1, arr2)`
- `np.mod()`: Element-wise modulo operation.
Example: `arr_mod = np.mod(arr1, arr2)`
- `np.power()`: Element-wise power of arrays.
Example: `arr_pow = np.power(arr1, 2)`
- `np.sqrt()`: Element-wise square root of array elements.
Example: `arr_sqrt = np.sqrt(arr)`
- `np.exp()`: Element-wise exponential of array elements.
Example: `arr_exp = np.exp(arr)`
- `np.log()`: Natural logarithm of array elements.
Example: `arr_log = np.log(arr)`
- `np.log10()`: Logarithm base 10 of array elements.
Example: `arr_log10 = np.log10(arr)`
- `np.abs()`: Compute the absolute value of array elements.
Example: `arr_abs = np.abs(arr)`

- `np.round()`: Round the elements of an array to the nearest integer.

Example: `arr_rounded = np.round(arr, 2)`

- `np.sign()`: Sign of array elements (1, -1, or 0).

Example: `arr_sign = np.sign(arr)`

- `np.clip()`: Limit the values in an array to a specified range.

Example: `arr_clipped = np.clip(arr, 0, 10)`

5. Statistical Methods

- `np.mean()`: Compute the mean of an array.

Example: `mean_val = np.mean(arr)`

- `np.median()`: Compute the median of an array.

Example: `median_val = np.median(arr)`

- `np.std()`: Compute the standard deviation of an array.

Example: `std_val = np.std(arr)`

- `np.var()`: Compute the variance of an array.

Example: `var_val = np.var(arr)`

- `np.min()`, `np.max()`: Compute the minimum and maximum of an array.

Example: `min_val = np.min(arr)`, `max_val = np.max(arr)`

- `np.percentile()`: Compute the nth percentile of an array.

Example: `percentile_50 = np.percentile(arr, 50)`

- `np.argmin()`, `np.argmax()`: Get the index of the minimum and maximum elements in an array.

Example: `min_idx = np.argmin(arr)`, `max_idx = np.argmax(arr)`

- `np.sum()`: Compute the sum of the array elements.

Example: `arr_sum = np.sum(arr)`

- `np.prod()`: Compute the product of the array elements.

Example: `arr_prod = np.prod(arr)`

- `np.cumsum()`: Compute the cumulative sum of array elements.

Example: `arr_cumsum = np.cumsum(arr)`

- `np.cumprod()`: Compute the cumulative product of array elements.

Example: `arr_cumprod = np.cumprod(arr)`

- `np.all()`, `np.any()`: Check if all or any elements of the array are True.

Example: `all_true = np.all(arr)`, `any_true = np.any(arr)`

- `np.diff()`: Compute the n-th discrete difference of an array.

Example: `arr_diff = np.diff(arr)`

- `np.unique()`: Find the unique elements of an array.

Example: `arr_unique = np.unique(arr)`

6. Linear Algebra

- `np.dot()`: Compute the dot product of two arrays.

Example: `dot_product = np.dot(arr1, arr2)`

- `np.matmul()`: Matrix multiplication.
Example: `result = np.matmul(arr1, arr2)`
 - `np.linalg.inv()`: Compute the inverse of a square matrix.
Example: `arr_inv = np.linalg.inv(arr)`
 - `np.linalg.det()`: Compute the determinant of a matrix.
Example: `det_val = np.linalg.det(arr)`
 - `np.linalg.eig()`: Compute the eigenvalues and eigenvectors of a square matrix.
Example: `eigvals, eigvecs = np.linalg.eig(arr)`
 - `np.linalg.svd()`: Compute the Singular Value Decomposition (SVD) of a matrix.
Example: `U, S, V = np.linalg.svd(arr)`
 - `np.linalg.norm()`: Compute the norm (magnitude) of an array.
Example: `arr_norm = np.linalg.norm(arr)`
-

7. Broadcasting and Advanced Indexing

- `arr[:, :]`: Slicing array (select rows and columns).
Example: `arr[1:3, 2:4]`
- `arr[condition]`: Apply a condition to index elements.
Example: `arr[arr > 2]`
- `arr[np.newaxis]`: Add a new axis (expand dimensions).
Example: `arr_expanded = arr[:, np.newaxis]`

- `arr[::-1]`: Reverse an array.

Example: `arr_reversed = arr[::-1]`

Numpy is a foundational library for numerical computing in python.

why it's is invaluable:

1. Efficiency
2. Mathematical Power (e.g, Linear algebra, statistics, trigonometry)
3. Foundation of Other Libraries
4. Broadcasting means ability to perform operations on array of different shapes without explicit looping simplifies code and boosts performance.
5. Data Manipulation

Array Creation

Numpy's core objective is the ndarray, a multi-dimensional array.

```
In [2]: #1.Array Creation
#np.array(obj,dtype=None)
import numpy as np
arr=np.array([1,2,3,4,5])
print(arr,type(arr))
print("Two dimensional Array:")
arr_2d=np.array([[1,3,5,6,7],[2,4,5,7,9]],dtype=str)
print(arr_2d)
```

```
[1 2 3 4 5] <class 'numpy.ndarray'>
Two dimensional Array:
[['1' '3' '5' '6' '7']
 ['2' '4' '5' '7' '9']]
```

```
In [13]: #np.zeros() creates an array filled with zero. shape is a tuple (row, column)
arr=np.zeros((2,3)) #np.zeros(shape, dtype=None)
print(arr)
```

```
[[0. 0. 0.]
 [0. 0. 0.]]
```

```
In [15]: #np.ones() creates an array filled with ones. shape is a tuple (row, column)
arr=np.ones((2,3)) #np.ones(shape,dtype=None)
print(arr)
```

```
[[1. 1. 1.]
 [1. 1. 1.]]
```

```
In [18]: #np.arange(start, stop, step, dtype=None) creates an array with evenly spaced values
arr=np.arange(0,100,2)
print(arr)
```

```
[ 0  2  4  6  8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46
 48 50 52 54 56 58 60 62 64 66 68 70 72 74 76 78 80 82 84 86 88 90 92 94
 96 98]
```

```
In [27]: #np.linspace() creates an array of num evenly spaced values between start and stop
arr=np.linspace(0,15,10) #np.linspace(start, stop, num=10, dtype=None)
print(arr)
```

```
[ 0.          1.66666667  3.33333333  5.          6.66666667  8.33333333
 10.         11.66666667 13.33333333 15.         ]
```

```
In [30]: #arr=np.eye() creates an identity matrix with 1s on the diagonal and 0s elsewhere
arr=np.eye(3) #np.eye(n,m=None, dtype=float)
print(arr)
```

```
[[1.  0.  0.]
 [0.  1.  0.]
 [0.  0.  1.]]
```

```
In [32]: arr=np.eye(3,4,dtype=float)
print(arr)
```

```
[[1.  0.  0.  0.]
 [0.  1.  0.  0.]
 [0.  0.  1.  0.]]
```

```
In [64]: #np.random.rand() creates an array of random values from a uniform distribution
arr=np.random.rand(2,2)
arr
```

```
Out[64]: array([[0.26639204, 0.02092549],
               [0.1345415 , 0.78463669]])
```

```
In [4]: #np.random.randn() creates an array of random values from a standard normal distribution
arr=np.random.randn(2,2)
print(arr)
```

```
[[-0.14536217 -0.19431646]
 [ 0.99470441 -0.27883956]]
```

```
In [38]: #np.random.randint(low, high=None, size=None, dtype=int) generates random integers
random_number=np.random.randint(1,7)
print(random_number)
```

```
4
```

```
In [63]: #np.random.choice(a, size=None, replace=True, p=None) generates random samples from an array
generated_sample=np.random.choice([1,3,4,6,9],size=2) #size specifies how many samples to generate
print(generated_sample)
```

```
[6 4]
```

```
In [64]: #practice
import numpy as np

# np.array(): Create from a list
arr1 = np.array([1, 2, 3, 4])
print("np.array():", arr1)
```

```

# np.zeros(): Array of zeros (2x3)
arr2 = np.zeros((2, 3))
print("np.zeros():", arr2)

# np.ones(): Array of ones (3x2)
arr3 = np.ones((3, 2))
print("np.ones():", arr3)

# np.arange(): Range from 0 to 10, step 2
arr4 = np.arange(0, 10, 2)
print("np.arange():", arr4)

# np.linspace(): 5 values from 0 to 1
arr5 = np.linspace(0, 1, 5)
print("np.linspace():", arr5)

# np.eye(): 3x3 identity matrix
arr6 = np.eye(3)
print("np.eye():", arr6)

# np.random.rand(): 2x2 random array (uniform 0 to 1)
arr7 = np.random.rand(2, 2)
print("np.random.rand():", arr7)

# np.random.randn(): 2x3 random array (normal distribution)
arr8 = np.random.randn(2, 3)
print("np.random.randn():", arr8)

# np.random.randint(): 5 random integers from 1 to 9
arr9 = np.random.randint(1, 10, size=5)
print("np.random.randint():", arr9)

# np.random.choice(): 3 random picks from [10, 20, 30]
arr10 = np.random.choice([10, 20, 30], size=3)
print("np.random.choice():", arr10)

```

```

np.array(): [1 2 3 4]
np.zeros(): [[0. 0. 0.]
 [0. 0. 0.]]
np.ones(): [[1. 1.]
 [1. 1.]
 [1. 1.]]
np.arange(): [0 2 4 6 8]
np.linspace(): [0. 0.25 0.5 0.75 1. ]
np.eye(): [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
np.random.rand(): [[0.34590198 0.82432524]
 [0.71554041 0.38519651]]
np.random.randn(): [[ 1.36752605 0.21560799 -0.19748942]
 [-0.87630366 0.75690177 1.02798767]]
np.random.randint(): [3 9 7 1 3]
np.random.choice(): [20 10 30]

```

Array Properties

Numpy arrays has attributes that let you inspect their structure and contents. These are not functions (no parentheses), but properties you can access directly from the array object.

```
In [65]: #arr.shape returns a tuple representing the array's dimensions (rows, columns)  
arr=np.array([[1, 2, 3], [4, 5, 6]])  
print(arr.shape)
```

```
(2, 3)
```

```
In [68]: #arr.size return the total number of elements in the array.  
print(arr.size)  
#arr.ndim return number of dimensions of the array.  
print(arr.ndim)  
#arr.dtype returns the data type of the array's elements.  
print(arr.dtype)
```

```
6
```

```
2
```

```
int32
```

```
In [70]: #arr.itemsize returns the size (in bytes) of each element in the array.  
print(arr.itemsize)
```

```
4
```

```
In [71]: #arr.T returns the transposed array  
print(arr.T)
```

```
[[1 4]  
 [2 5]  
 [3 6]]
```

```
In [72]: #arr.flatten() returns a 1D copy of the array, collapsing all dimensions.  
print(arr.flatten())
```

```
[1 2 3 4 5 6]
```

```
In [79]: #arr.reshape(shape) return a new array with specified shape.  
arr=arr.flatten()  
print(arr.reshape(2,3))
```

```
[[1 2 3]  
 [4 5 6]]
```

```
In [82]: #practice  
import numpy as np  
  
# Create a 2D array  
arr = np.array([[1, 2, 3], [4, 5, 6]], dtype=np.int32)  
  
print("Shape:", arr.shape)  
  
print("Size:", arr.size)
```

```

print("Dimensions:", arr.ndim)

print("Data type:", arr.dtype)

print("Item size (bytes):", arr.itemsize)

# arr.T: Transpose the array
print("Transpose:\n", arr.T)

print("Flattened:", arr.flatten())

print("Reshaped to 3x2:\n", arr.reshape(3, 2))

```

```

Shape: (2, 3)
Size: 6
Dimensions: 2
Data type: int32
Item size (bytes): 4
Transpose:
[[1 4]
 [2 5]
 [3 6]]
Flattened: [1 2 3 4 5 6]
Reshaped to 3x2:
[[1 2]
 [3 4]
 [5 6]]

```

Array Manipulations

These numpy functins let you modify, combine, or resturcture arrays.

```

In [86]: #np.concatenate((arr1,arr2,..),axis=0)
import numpy as np
arr1 = np.array([1, 2])
arr2 = np.array([3, 4])
result=np.concatenate((arr1,arr2),axis=0)
print(result)

```

```
[1 2 3 4]
```

```

In [94]: #np.stack((arr1,arr2,..), axis=0) stacks array along a new axis, creating a
result=np.stack((arr1,arr2))
print(result)
result1=np.hstack((arr1,arr2))
print("horizonatal stack:\n",result)
result=np.vstack((arr1,arr2))
print("vertical stack:\n",result)

```

```

[[1 2]
 [3 4]]
horizonatal stack:
[[1 2]
 [3 4]]
vertical stack:
[[1 2]
 [3 4]]

```

```

In [107... #np.split(array, indices_or_sections, axis=0)
arr=np.array([1, 2, 3, 4, 5, 6])
result=np.split(arr,3)
print(result)

```

```

[array([1, 2]), array([3, 4]), array([5, 6])]

```

```

In [109... #np.append(array, obj, values, axis=None)
result=np.append(arr,[2,4,5],axis=0)
print(result)

```

```

[1 2 3 4 5 6 2 4 5]

```

```

In [110... #np.insert(array, object, values, axis=None)
result=np.insert(arr, 1,12323)
print(result)

```

```

[   1 12323    2    3    4    5    6]

```

```

In [113... #np.delete(array, obj, axis=None)
result=np.delete(arr, 1) #Here 1 is index
print(result)

```

```

[1 3 4 5 6]

```

```

In [115... #np.resize(arr, new_shape) resizes an array to a new shape, repeating or truncating
result=np.resize(arr,(4,4))
print(result)

```

```

[[1 2 3 4]
 [5 6 1 2]
 [3 4 5 6]
 [1 2 3 4]]

```

```

In [116... #np.repeat(arr, repeats, axis=None)
result=np.repeat(arr,2)
print(result)

```

```

[1 1 2 2 3 3 4 4 5 5 6 6]

```

```

In [117... #np.tile(array, reps) repeats the entire array a specific number of times.
result=np.tile(arr,2)
print(result)

```

```

[1 2 3 4 5 6 1 2 3 4 5 6]

```

Mathematical Operatiions

These functions perform element-wise mathematical operations on arrays, which are key for data transformations

```
In [118... import numpy as np
arr1 = np.array([1, 2])
arr2 = np.array([3, 4])
result = np.add(arr1, arr2)
print(result)
```

```
[4 6]
```

```
In [120... print(np.subtract(arr1, arr2))
```

```
[-2 -2]
```

```
In [121... print(np.multiply(arr1, arr2))
```

```
[3 8]
```

```
In [122... print(np.divide(arr1, arr2))
```

```
[0.33333333 0.5      ]
```

```
In [123... print(np.mod(arr1, arr2))
```

```
[1 2]
```

```
In [124... print(np.power(arr1, 3))
```

```
[1 8]
```

```
In [125... print(np.sqrt(arr1))
```

```
[1.         1.41421356]
```

```
In [126... np.exp(arr1)
```

```
Out[126... array([2.71828183, 7.3890561 ])
```

```
In [127... np.log(arr1)
```

```
Out[127... array([0.         , 0.69314718])
```

```
In [128... np.log10(arr1)
```

```
Out[128... array([0.         , 0.30103])
```

```
In [131... arr = np.array([-1, -2])
np.abs(arr1)
```

```
Out[131... array([1, 2])
```

```
In [132... arr = np.array([1.55, 2.45])
np.round(arr, 1)
```

```
Out[132... array([1.6, 2.4])
```

```
In [133... arr=np.array([-2,0,3])
np.sign(arr)
```

```
Out[133... array([-1,  0,  1])
```

```
In [134... #np.clip(arr, a_min, a_max) limits elements to a specific range
arr=np.array([1,2,5,10,67])
np.clip(arr, 2,7)
```

```
Out[134... array([2, 2, 5, 7, 7])
```

Statistical Methods

```
In [151... import numpy as np
arr = np.array([1, 2, 3])
np.mean(arr)
```

```
Out[151... 2.0
```

```
In [152... print("Median:", np.median(arr))
print("Standard Deviation:", np.std(arr))
print("Variance:", np.var(arr))
print("Minimum:", np.min(arr))
print("Maximum:", np.max(arr))
print("25th Percentile:", np.percentile(arr, 25, axis=0))
print("75th Percentile:", np.percentile(arr, 75, axis=0))
print("Mean:", np.mean(arr))
```

```
Median: 2.0
Standard Deviation: 0.816496580927726
Variance: 0.6666666666666666
Minimum: 1
Maximum: 3
25th Percentile: 1.5
75th Percentile: 2.5
Mean: 2.0
```

```
In [153... print(np.argmin(arr))
print(np.argmax(arr)) #these both return index
```

```
0
2
```

```
In [154... print("sum: ",np.sum(arr))
print("product: ", np.product(arr))
```

```
sum: 6
product: 6
```

```
In [155... np.cumsum(arr)
```

```
Out[155... array([1, 3, 6])
```

```
In [156... np.cumprod(arr)
```

```
Out[156... array([1, 2, 6])
```

```
In [158... np.all([0,2])
```

```
Out[158... False
```

```
In [159... np.any([0,2])
```

```
Out[159... True
```

```
In [161... np.diff(arr)
```

```
Out[161... array([1, 1])
```

```
In [163... np.unique(arr)
```

```
Out[163... array([1, 2, 3])
```

Statistical Methods

Numpy's **linalg** module provides tools for matrix operations

```
In [165... import numpy as np
arr1 = np.array([1, 2])
arr2 = np.array([3, 4])
print(np.dot(arr1,arr2))
```

```
11
```

```
In [168... arr1 = np.array([[1, 2], [3, 4]])
arr2 = np.array([[5, 6], [7, 8]])
print(np.matmul(arr1,arr2))
np.dot(arr1,arr2)
```

```
[[19 22]
 [43 50]]
```

```
Out[168... array([[19, 22],
 [43, 50]])
```

```
In [170... result=np.linalg.inv(arr1)
print(result)
```

```
[[-2.   1. ]
 [ 1.5 -0.5]]
```

```
In [171... np.linalg.det(arr1)
```

```
Out[171... -2.0000000000000004
```

```
In [174... eigvals, eigvecs = np.linalg.eig(arr1)
print(f"{eigvals}\n\n{eigvecs}")
```

```
[-0.37228132  5.37228132]
```

```
[[-0.82456484 -0.41597356]  
 [ 0.56576746 -0.90937671]]
```

```
In [175... #single value decomposition  
u,s,v=np.linalg.svd(arr1)  
print(u,s,v)
```

```
[[-0.40455358 -0.9145143 ]  
 [-0.9145143  0.40455358]] [5.4649857  0.36596619] [[-0.57604844 -0.8174155  
6]  
 [ 0.81741556 -0.57604844]]
```

```
In [182... arr=np.array([3,4])  
np.linalg.norm(arr)
```

```
Out[182... 5.0
```

Broadcasting and Advanced Indexing

NumPy's broadcasting and indexing features allow efficient operations on arrays of different shapes and precise data selection.

```
In [183... #arr[start:stop:step, start:stop:step] slice rows and columns of an array.  
arr=np.array([[1,2,3],[4,5,6]])  
result=arr[0:2,1:3]  
print(result)
```

```
[[2 3]  
 [5 6]]
```

```
In [184... #arr[condition]  
result=arr[arr>2]  
print(result)
```

```
[3 4 5 6]
```

```
In [186... #reversing  
arr[::-1]
```

```
Out[186... array([[4, 5, 6],  
                [1, 2, 3]])
```